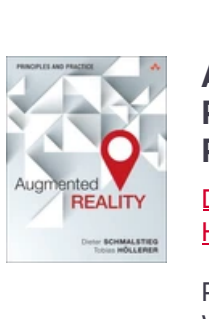


Augmented Reality: Principles and Practice
Dieter Schmalstieg, Tobias Höllerer
Published by Addison-Wesley Professional
0% complete Approx. 16 hours left

Collapse

Contents Highlights

Temporal Coherence
Image-Guided Placement
Legibility
X-Ray Visualization
Spatial Manipulation
Information Filtering
Summary
Chapter 8. Interaction
Chapter 9. Modeling And Annotation
Chapter 10. Authoring
Chapter 11. Navigation
Chapter 12. Collaboration
Chapter 13. Software Architectures

Chapter 7. Situated Visualization

In Chapter 6, we examined *how* computer graphics can be seamlessly embedded in a real scene, while we silently assumed that it is well known *what* information should be presented in AR. In truth, this is an important problem. The power of AR as a novel user interface paradigm derives in large part from its ability to display information that is relevant to a situation, task, or user. To successfully communicate the desired information, it must be put in an appropriate visual form. This is done through the application of suitable visualization techniques.

Visualization for AR differs from conventional visualization primarily by its interaction with the real scene, which becomes an intrinsic part of the visualization [Kalkofen et al. 2011]. In this chapter, we investigate **situated visualization** techniques that somehow take the real world into account.

The term *situated visualization* was coined by White and Feiner [2009] to describe a form of context-aware computing, where the context is defined as part of the physical scene. Such a context does not necessarily have to be a single object. It can be anything from an isolated point in space (Figure 7.1) to a large area such as a particular city district, as long as the artifact has some semantic meaning in the real world.

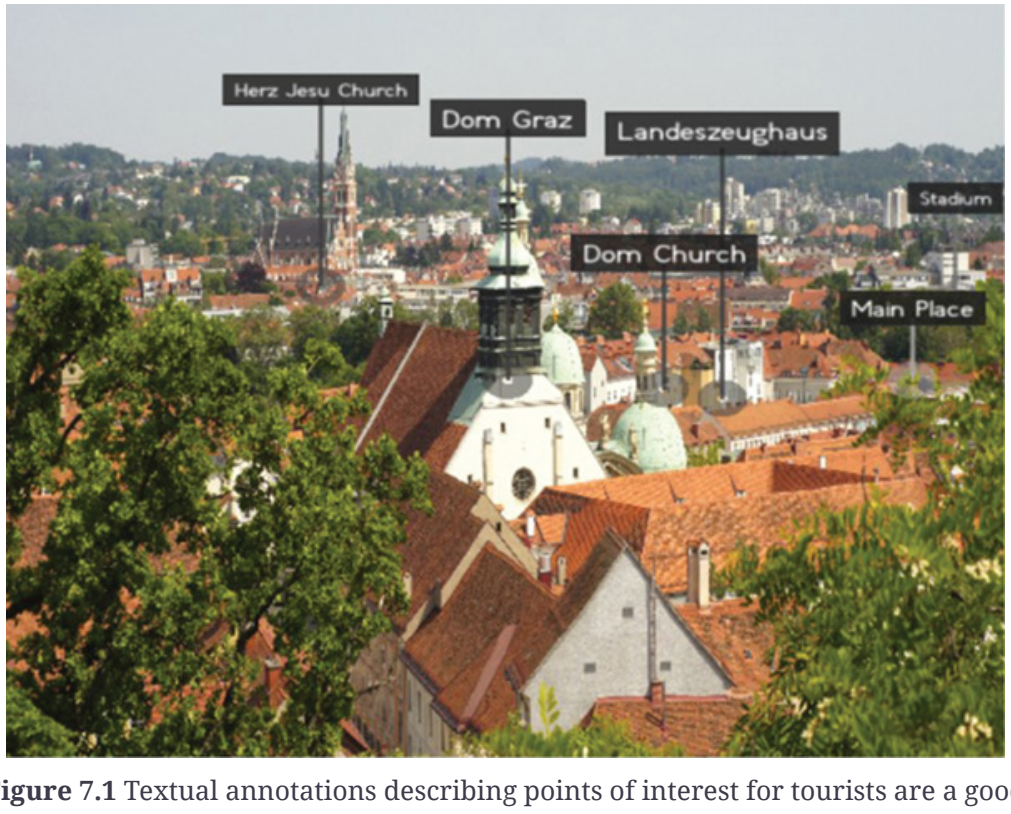


Figure 7.1 Textual annotations describing points of interest for tourists are a good example of situated visualization. Courtesy of Raphael Grasset.

What does not count as situated are visualizations that are merely registered in three-dimensional space, usually as a result of deploying a particular tracking technology. For example, many AR applications that use fiducial markers as a tracking aid register their virtual content relative to a particular marker (Figure 7.2). The marker is obviously a physical object but does not have any significance in the real world other than to facilitate tracking. The AR application can be transported to an arbitrary alternate location without affecting its semantics, which is why it cannot be called situated. This type of visualization is not significantly different from visualization in VR and is not discussed further in this chapter.

→ No cuentan visualizaciones situadas en un espacio 3D sin significado real para el mundo físico

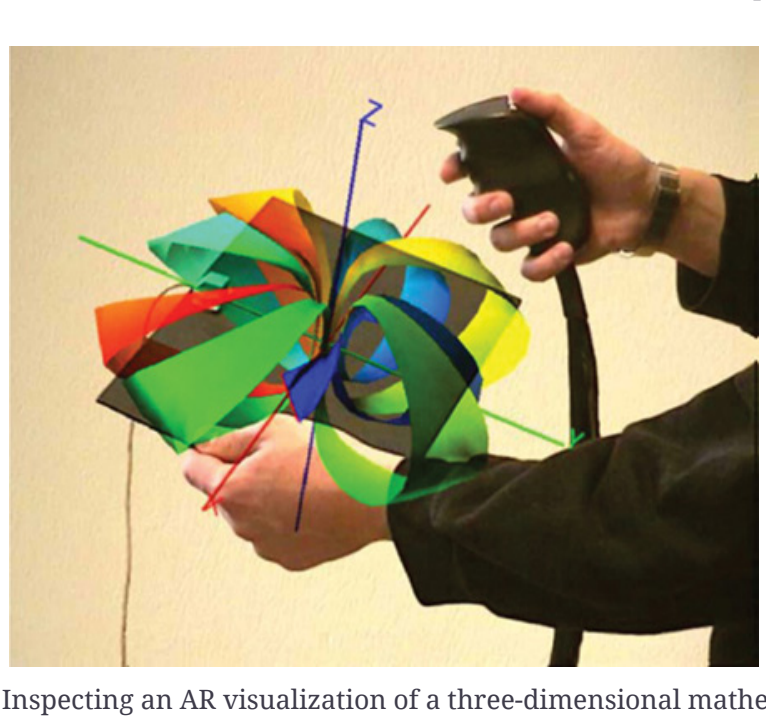


Figure 7.2 Inspecting an AR visualization of a three-dimensional mathematical model is a compelling application, but not a situated visualization, because no semantically significant real-world object is referred to. Courtesy of Anton Fuhrmann.

We start our investigation of situated visualization by considering the challenges that this type of visualization must meet. This examination provides us with an understanding of the design problem we face. The remaining sections deal with one important design idea each. We first discuss how to address fundamental registration issues. This is followed by an exploration of approaches for labeling and laying out information in AR. We then consider the use of X-ray visualization to convey hidden information. The following section outlines an alternative, spatial manipulation of the scene, which can be used to uncover information, if X-ray visualization is not desired. Finally, we discuss various forms of information filtering that make it possible to handle scenes that are densely populated with information.

Challenges

Situated visualization faces the same challenges as traditional visualization, including the same primary challenge—*data overload*. When too much data is presented, it leads to a cluttered presentation and impairs understanding. To build the necessary understanding, users also must be able to explore the data *interactively*.

In addition, the embedding of situated visualizations into the real world requires addressing several other issues. *Registration errors* affecting the placement of the visualization may lead to the communication of false information. Even if registration is perfect, a viewer's ability to easily discern important from irrelevant information can be reduced by *visual interference* between real and virtual objects. Moreover, the real world (including the user's viewpoint) is constantly changing and requires adjustments to the visualization. To avoid distractions, the adjustments must be performed in a way that ensures *temporal coherence*. In the following subsections, we discuss each of these challenges in more detail.

Data Overload

Presenting a large amount of data in AR quickly leads to a cluttered presentation, which makes it difficult for a viewer to gain insight into the data. This problem is aggravated in AR, which often has a limited presentation space, such as a smartphone display. Azuma et al. [2001] identified this situation as the problem of increasing data density. They refer to two complementary solutions to manage the data. The first solution is to reduce the amount of data by **filtering** [Feiner et al. 1993a] [Julier et al. 2002]. The second solution is to create a visualization **layout** that avoids interference with other important information. Spence [2007] refers to the issue of data density in information visualization as data overload.

→ solución

Early on, visualization research developed architectural models to cope with this problem [Haber and McNabb 1990] [Card et al. 1999]. These models generally incorporate three steps: *data transformation*, *visual mapping*, and *view transformation* (Figure 7.3).

- Data transformation reduces the amount of data by filtering or aggregating data points.
- Visual mapping creates visual structures of the data such as color and shape.
- View transformation determines properties such as the position and scale of the visual structures, addressing the layout problem.

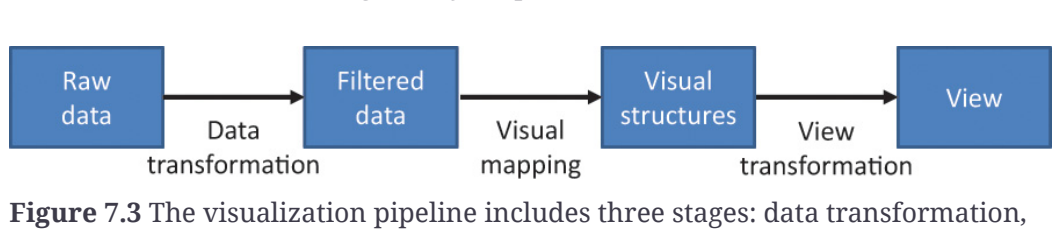


Figure 7.3 The visualization pipeline includes three stages: data transformation, visual mapping, and view transformation.

User Interaction

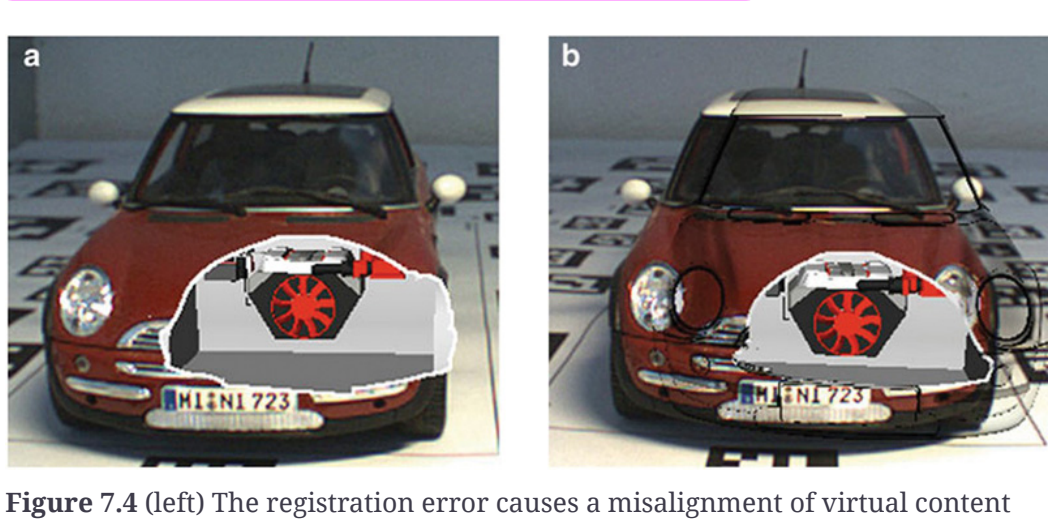
In contrast to hand-drawn illustrations, a core aspect of computer-supported visualization is the ability to interactively explore the data. In the area of information visualization, the design of user interfaces was heavily influenced by the "information seeking mantra" of Shneiderman [1996]:

- Overview: Gain an overview of the entire collection
- Zoom: Zoom in on items of interest
- Filter: Filter out uninteresting items
- Details-on-demand: Select an item or group and get details when needed

While Shneiderman did not intend these steps to be prescriptive [Craft and Cairns 2005], many successful visualizations have been designed by following this mantra. Therefore, designers of situated visualizations should also consider these recommendations, as they will likely support users getting valuable insights into the data.

Registration Errors

Situated visualizations have an additional requirement relative to conventional visualizations—namely, registration to an artefact that is present in the real world. Tracking errors can lead to inaccurate registration, causing misalignments between the augmentations and the real-world objects they refer to. To cope with this problem, visualizations should take the precision of the registration into account. For instance, the error can be communicated to the viewer by integrating a virtual copy of the real-world context into the visualization (Figure 7.4).



→ Se busca compensar el error de alguna forma, como esta

Figure 7.4 (left) The registration error causes a misalignment of virtual content and the real-world context—in this case, the engine compartment. (right) The registration error can be resolved by integrating a copy of the context—in this case, the outlines of the car—into the visualization. Courtesy of Denis Kalkofen.

Visual Interference

Visualizations generally emphasize the relevant parts of data in a way that guides the viewer's attention to this information. Without this emphasis, such information might be overlooked, because it does not stand out from the rest of the data. In situated visualization, the viewer's attention is guided toward the important parts of the scene, while irrelevant aspects of the real world are not distracting. Achieving this outcome requires avoiding visual interferences between the visualization and the real world. Kalkofen et al. [2007] identified this challenge as a focus and context visualization problem in AR: The relevant part of the scene is the focus, while the rest of the scene provides context. Figure 7.5 shows that the focus is emphasized to guide the viewer's attention and the rest of the scene is deemphasized to avoid distractions.

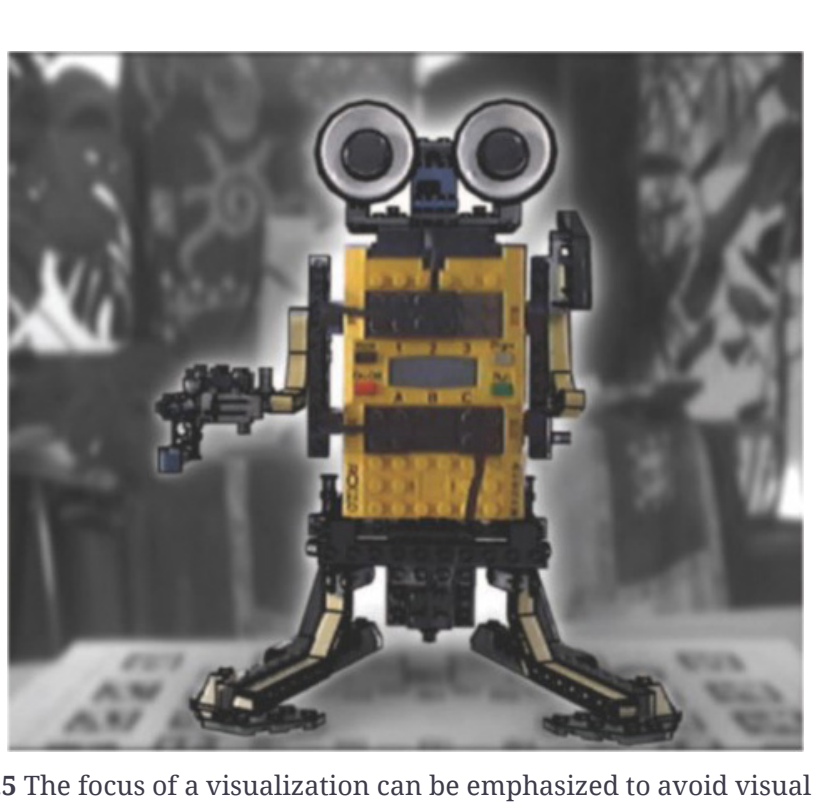


Figure 7.5 The focus of a visualization can be emphasized to avoid visual interferences with the background and to guide the viewer's attention. Courtesy of Denis Kalkofen.

Visual interferences between the visualization and the real world can also be caused by the placement of the augmentations, which can cause occlusions of the augmented real-world object and other important landmarks. View management techniques have been developed that prevent such occlusions by rearranging the virtual content [Bell et al. 2001] [Grasset et al. 2012].

Temporal Coherence

A major difference of AR visualization over conventional visualization is that the real-world context is not static, but rather changes over time. For instance, people or cars may pass through the video image, or the lighting conditions may change. Consider the visualization of the internals of a car in Figure 7.6. While the internals are clearly visible in the yellow car, the same visualization is obscured when the car is red.

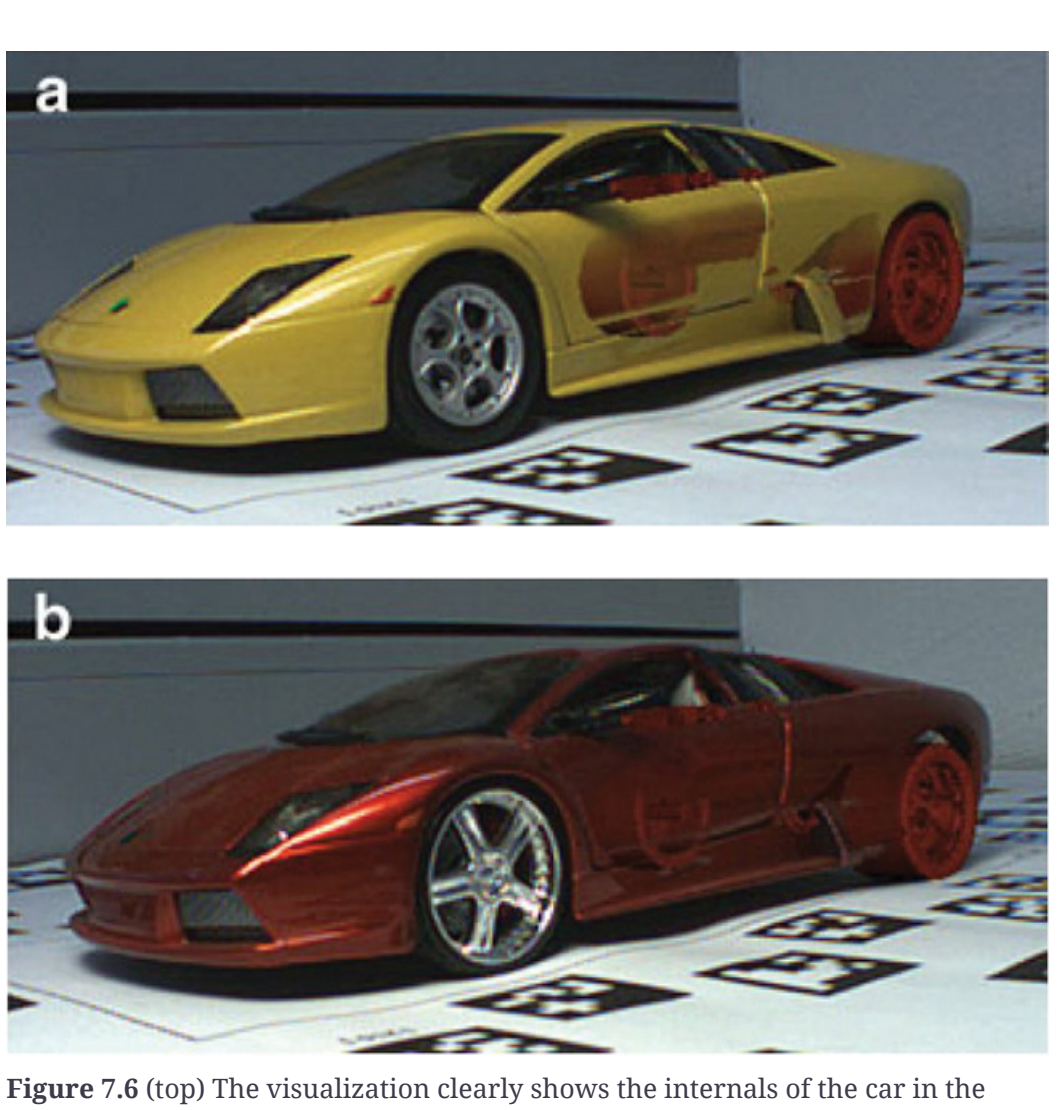


Figure 7.6 (top) The visualization clearly shows the internals of the car in the rear. (bottom) A poor choice of color severely impacts the perception of the occluded visualization. Courtesy of Denis Kalkofen.

Hence, static augmentations may be effective for a certain amount of time, but then cease to be effective at a later time. If we want to ensure consistent effectiveness, situated visualizations must be able to *adapt* to changing environmental conditions. However, frequent and strong adaptations are also undesirable, because they can distract the viewer. Therefore, visualizations must behave in a *temporally coherent* way. This aspect is especially important for constrained layouts, where simple viewpoint changes can readily introduce significant changes.

Visualization Registration

We start the discussion of methods leading to situated visualization with the issue of registration. Recall that situated visualization must refer to a real-world artefact, which may or may not be a moveable object. We can distinguish two situations on this basis:

- The visualization is given in local coordinates of a potentially moveable object. → objeto movible
- The visualization is given in global coordinates relative to a stationary object or physical location. → 〃 estatico o lugar

These two options lead to different types of visualization design.

Locally Registered Situated Visualization

In the local case, to apply the situated visualization, we must first detect the presence of the object referred to or otherwise establish the local coordinate system. When this detection step relies on dynamic object recognition—for example, using image search—the artifact referred to might potentially not be unique. This ambiguity can be intentional. For example, advertising can be expressed relative to a particular brand logo. Whenever that logo occurs, the situated visualization would be displayed. Another example of visualization relative to a dynamically detected object is documented by White et al. [2006], who describe virtual vouchers to support botanists in their fieldwork. This system is able to identify various types of leaves by image recognition when the leaf is placed on an instrument clipboard.

→ Ejemplo de las hojas!!!

Globally Registered Situated Visualization

If the visualization refers to a stationary artifact or a particular location, it can be expressed in absolute global coordinates. In this case, it is not strictly necessary to detect the referred object, as long as a global localization system is available. If no object detection is performed, however, there is a certain danger that the environment may have changed since the database was created, such that the relevant object is no longer there. A possible alternative to global localization, then, is to first establish a coarse global location (e.g., using GPS) and then search for objects known to be in the vicinity.

→ Riesgo de que si no se realiza inspección, el objeto o lugar haya desaparecido con el tiempo

One advantage of global references is that the situated visualization can easily be put in geometric relation to other globally registered visualizations. This relationship can become relevant for certain algorithms such as dynamic layout generation (see the section "Annotations and Labeling").

As an example for globally referenced visualization, consider the case of AR for subsurface infrastructure visualization [Schall et al. 2008]. Infrastructure providers such as utility companies maintain geospatial databases for their buried assets, such as power lines or gas pipes. For maintenance procedures, it is necessary to locate these assets in the field. Maintenance workers can navigate to a given location and observe a situated visualization of the relevant infrastructure. In this case, the visualized data is precisely referenced to global geographic coordinates, but not necessarily related to any (visible) object.

Another example where AR can visualize globally referenced information that is not tied to a particular physical object is sensor data visualization. Suppose environmentalists, urban planners, and other professionals regularly visit a site to collect impressions relevant for their professional activities. Sensors deployed throughout this environment, either by the professionals as part of their site visits or as part of a permanent monitoring infrastructure, might deliver important information on the environment—for example, air pollution or moisture levels. Rather than having to relate such sensor information to observations made on the site after returning to the planning office, it would be useful to be able to observe sensor data in situ—that is, directly in the environment where the data is collected.

This approach was explored by White and Feiner [2009] with SiteLens, a system that is designed to display carbon monoxide levels in New York City, with the data being obtained from both mobile and stationary sensors. Veas et al. [2012a] discuss Hydrosys, a system that displays hydrological information in the Swiss Alps, based on a widely deployed global sensor network. This network collects information on issues such as water levels or temperature, which is relevant for glacier monitoring (Figure 7.7).

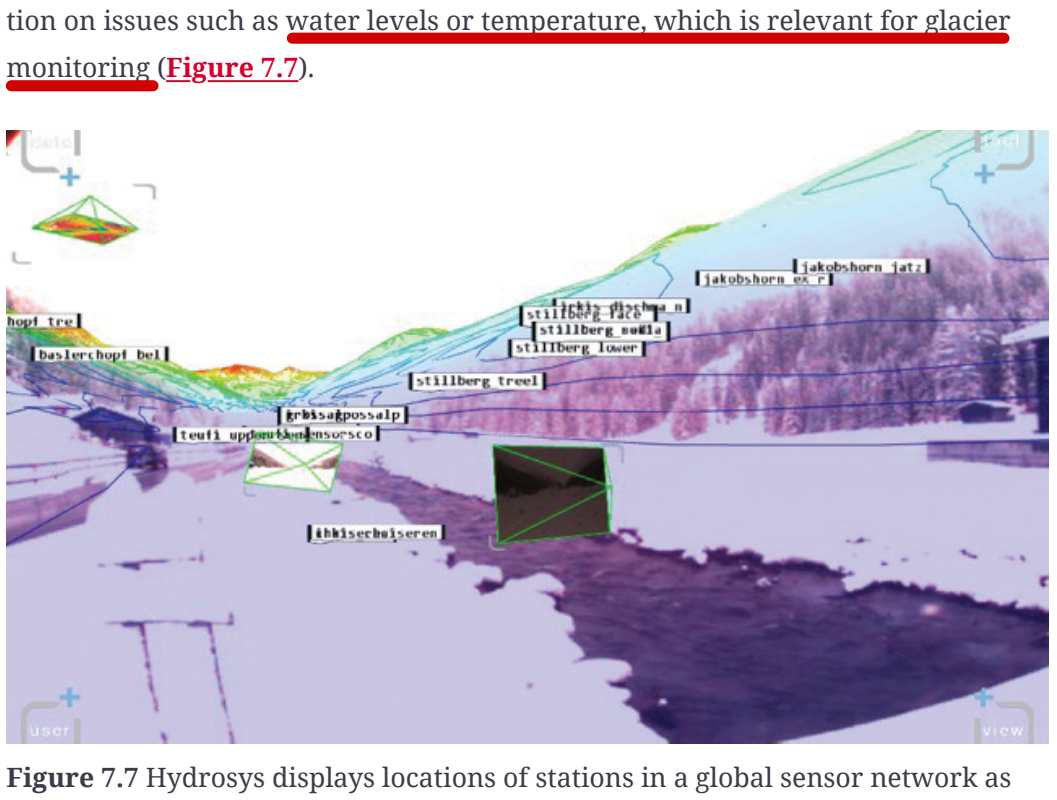


Figure 7.7 Hydrosys displays locations of stations in a global sensor network as well as interpolated temperature plotted as geodesic contours. Courtesy of Eduardo Veas and Ernst Kruijff.

Registration Uncertainty

Situated visualization requires precise registration of virtual and real scene elements. Two major sources of error can affect this registration. First, the position and orientation of virtual objects may not exactly match those of their real counterparts. Second, the position and orientation of the virtual camera may not exactly match those of the real camera. Both types of errors degrade the quality of the resulting visualization, because the overlaid graphics will not properly align with objects in the physical world. As a result, augmentations can be ambiguous or even misleading, if they point to the wrong object.

Fuentes de error:
1) objetos o referencias virtuales no van en línea con el objeto real
2) lo mismo pero para la cámara virtual y real

This problem can be partially addressed by making situated visualizations adaptive to registration uncertainty. Many tracking techniques enable estimation of the amount of tracking error. Using geometric relationships, as, for example, encoded in a scene graph, the error estimate can be transformed into screen-space error. Such a screen-space error can be used to parameterize visualizations so that they become more error tolerant.

→ Solución

MacIntyre et al. [2002] propose several types of error-tolerant visualizations. One such approach enlarges the silhouette of a phantom object such that the resulting screen-aligned shape is certain to cover the corresponding real object even in the presence of registration errors. Another idea is to dynamically switch textual labels placed on top of a real object to a representation that uses a leader line between label and object. The latter representation is more robust against registration errors and, therefore, is used when the registration error exceeds a certain threshold.

→ Una idea de sol.

→ otra sol. Más confiable.

Annotations and Labeling

One of the major advantages of situated visualization is its ability to present information close to real-world objects in the form of **annotations**. According to the definition given by Wither et al. [2009], an annotation must always be composed of a spatially dependent component, which anchors the object in the real world, and a spatially independent component, which introduces additional information not otherwise contained in the real world. This definition is very broad and encompasses many possible applications from the domain of interactive guidance systems, such as for navigation, tourism, or maintenance instructions. Perhaps the most important class of annotations comprises textual labels, which provide explanations or hints concerning real-world artifacts.

Labeling Fundamentals

Placement of textual labels has been widely studied in conventional 2D maps, but the 3D case turns out to be very different and more complicated. In this section, we will explain some fundamental considerations in these applications. First, we must first distinguish internal from external labels. Second, we require objectives for proper placement of labels. Third, we need to choose an appropriate representation of the screen space where labels are to be placed. Fourth, we must choose anchor points for any leader lines that are required.

Internal and External Labels

Two principal approaches to placing a label are possible: internal and external. Internal labels are placed directly over the object—that is, within its silhouette. If the object is partially occluded, the label should be placed over the visible portion to avoid any ambiguities. In contrast to internal labels, external labels are placed outside the object's silhouette but should be in the object's vicinity to avoid any ambiguities. External labels use a leader line to link the label to an anchor point inside the silhouette of the object to which that label refers.

Placement Objectives

The placement of labels over a view of a three-dimensional scene should follow a number of widely accepted criteria:

- Labels should be placed close to the referred objects.
- Labels should not overlap.
- External labels should not be placed over other important objects.
- The length of each leader line should be minimal.
- Crossings of leader lines with each other and with labels are undesirable.
- Temporal coherence should be maintained—label positions should not abruptly change from one frame to the next.

Formulating these criteria mathematically leads to a constrained optimization problem, which is known to be NP-hard. However, solutions can be computed using heuristic optimization strategies. Several published approaches derive usual solutions using relatively simple greedy algorithms that find suitable placements for one label after the other, given some ordering of the labels [Azuma and Furmanski, 2003].

Area Representation

For solving the optimization problem, it is necessary to have a representation of the screen area that enables determination of whether a point is occupied by the object to be labeled, by another important object that should not be disturbed, or by a background (such as grass or sky) where labels and leader lines can freely be placed. Such an area representation can either be discrete, based on the screen-aligned bounding boxes of objects [Bell et al. 2001], or sampled, being given as a two-dimensional array of arbitrary spacing (not necessarily identical to screen resolution). In both cases, hidden surface removal must be performed to correctly identify occlusions among objects.

A sampled representation can easily be produced on the GPU by rasterizing the scene objects into an id-buffer. Unfortunately, reading back the id-buffer can introduce significant latency. Therefore, Hartmann et al. [2004] perform the rasterization on the CPU, while Stein and Décoret [2008] execute the whole algorithm on the GPU.

Choosing the Anchor Point

If the anchor point to which a leader line should be attached is not given by the application, it should be determined in a suitable way. In general, the anchor point should be chosen at a surface position of the object that is visible in the current frame. For convex objects (or if only a bounding box of the object is available), the centroid is a reasonable choice. In contrast, for arbitrarily shaped objects, a more robust procedure is to iteratively apply morphological thinning and assign the anchor point to the last remaining position before the object completely disappears.

Optimization Techniques

With an appropriate understanding of the objectives for label placement, we can formulate placement as an optimization problem. Because the problem needs to be solved for every frame, it must be handled efficiently. Suitable optimization methods use either (1) a sparse formulation with forces or bounding boxes or (2) a dense formulation that is amenable for parallel evaluation on the GPU.

Optimization Using Forces

Hartmann et al. [2004] formulate label placement as an optimization problem carried out on force fields set up in image space. The force fields model the following forces applied to a label:

- An attractive force from an object's 2D projection to the screen space
- A repulsive force from the object boundary (so that a label is placed either fully internally or fully externally)
- A repulsive force from the projection of other objects
- A repulsive force from the screen boundary
- A repulsive force from other labels

All the force fields are combined using a weighted average. First, a heuristic is applied to initialize the label position. Second, labels follow the gradient of the combined force field, until a minimum is found. Interfering labels that overlap in the screen space are adjusted dynamically using a simple greedy heuristic that favors larger labels.

Leerlo Después

Optimization Using Bounding Boxes

Bell et al. [2001] approximate the 2D projection using screen-aligned 2D bounding boxes. The empty space remaining after subtracting the space occupied by the bounding boxes is considered as a site for placing the labels. The algorithm of Bell et al. begins by determining the size of the visible portion of the object to which the label will be attached. If it is large enough, an internal label can be placed; if not, the algorithm searches for a suitable site for placing an external label. External labels are placed by a greedy algorithm that prioritizes labels in front-to-back order but could also use other kinds of priorities. After placing a label, the occupied area is forbidden for other labels.

Optimization on the GPU

Stein and Décoret [2008] use the power of modern GPUs to implement a greedy real-time optimization that considers all possible placements for a label. The order in which labels are considered is based on the use of a Voronoi diagram to approximately sort the anchor points from inward (i.e., from the center of the scene) to outward (i.e., toward the screen boundary). This algorithm renders a texture representation of forbidden regions for the label, which considers important objects, other labels, and leader lines. Finally, a GPU program systematically tests all remaining locations and determines the optimum.

Temporal Coherence

If the optimization problem is solved for every frame independently, the placement of labels can vary significantly from frame to frame, resulting in occlusions (Figure 7.8) or a jumpy appearance (Figure 7.9). As a countermeasure, an additional hysteresis constraint can be introduced, which penalizes large label displacements. Moreover, if a label must be displaced over a larger distance, this transition can be animated over several consecutive frames for a smoother transition.

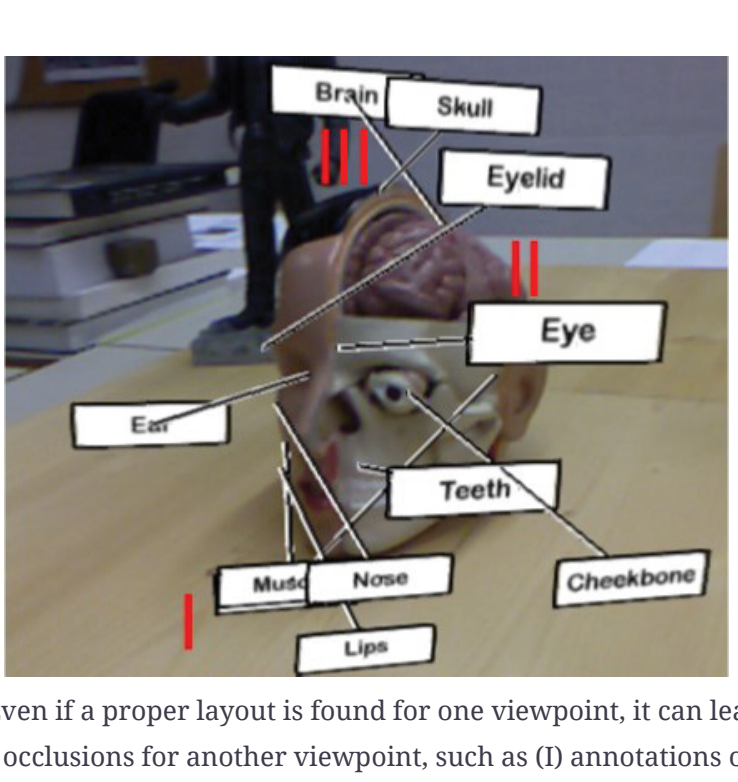


Figure 7.8 Even if a proper layout is found for one viewpoint, it can lead to various types of occlusions for another viewpoint, such as (I) annotations occluding each other, (II) annotations occluding the object of interest, or (III) leader lines crossing each other. Courtesy of Markus Tatzgern and Denis Kalkofen.

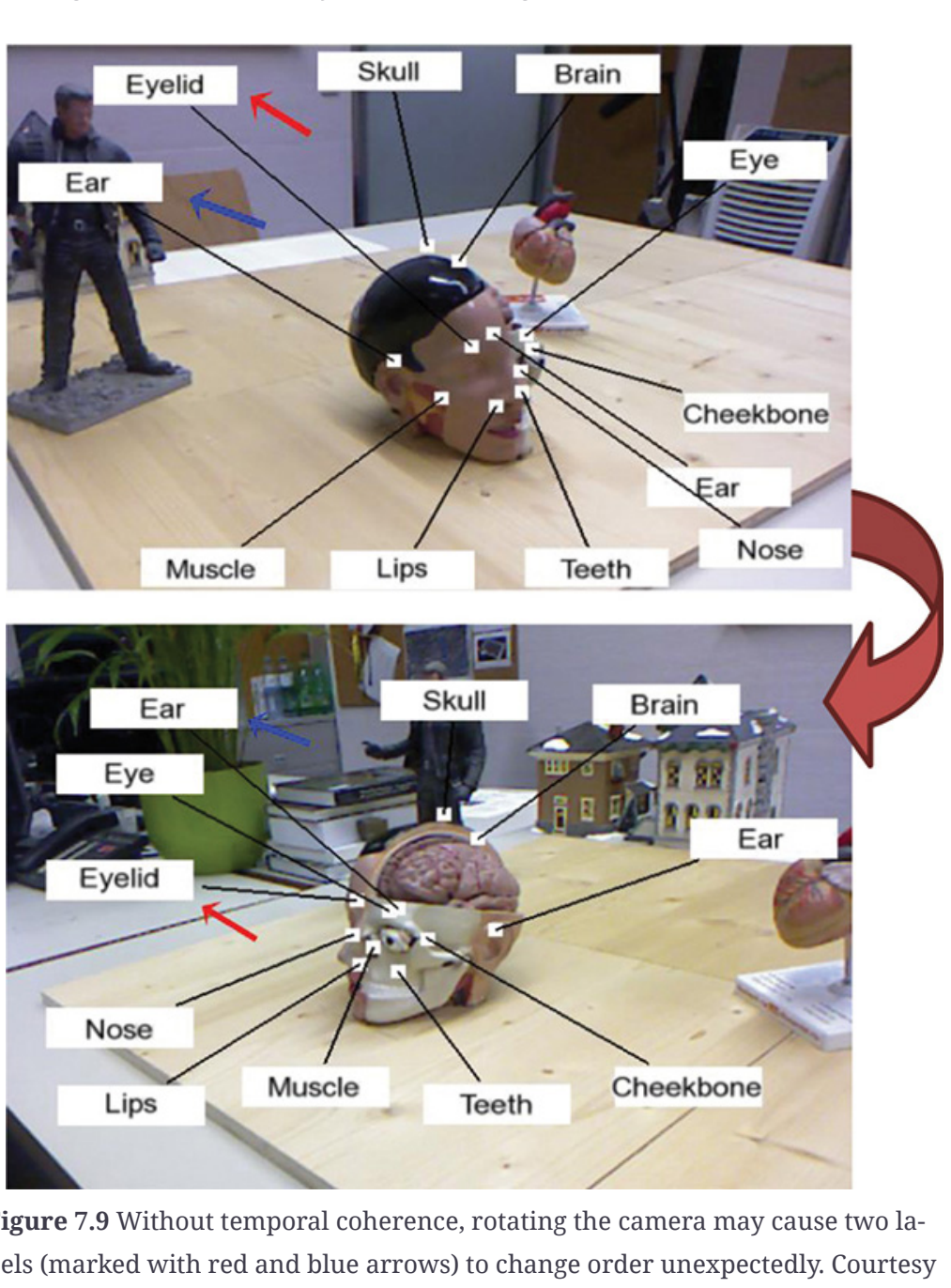


Figure 7.9 Without temporal coherence, rotating the camera may cause two labels (marked with red and blue arrows) to change order unexpectedly. Courtesy of Markus Tatzgern and Denis Kalkofen.

Label layout in 2D image space cannot address temporal coherence of objects are moving in a 3D scene. A 3D layout can address this issue more adeptly, because labels can follow object motion in world space [Pick et al. 2010]. A temporally coherent view management approach can use this idea to compute a constrained layout for a label with 3D geometry. A 3D label consists of a 3D annotation billboard, a 3D pole (the equivalent of a 2D leader line), and an anchor point on a real-world object. Only adjustments to the length of the pole and a small displacement of the annotation billboard in image space are allowed, while the orientation of the pole is fixed in object space. Figure 7.10 illustrates this strategy, which resembles a "hedgehog" [Tatzgern et al. 2014b]. Label placement has enough freedom to produce high-quality layouts without suffering from temporal coherence problems.

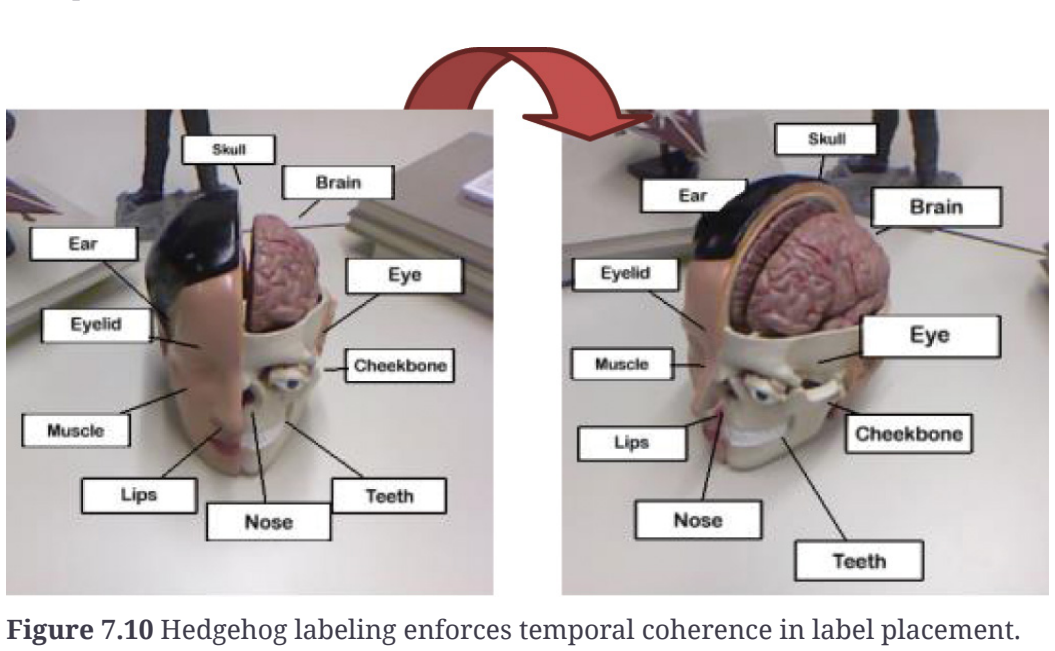


Figure 7.10 Hedgehog labeling enforces temporal coherence in label placement. Courtesy of Markus Tatzgern and Denis Kalkofen.

Image-Guided Placement

In AR, it will frequently be the case that only a subset of the objects in the environment is known to the application. In this case, the video image itself should be analyzed for areas of less visual interest, where labels can be placed without occluding salient content unknown to the application. To that end, Leykin and Tuceryan [2004] use a machine learning approach to automatically determine the readability of text over textured backgrounds. A similar goal is pursued by Rosten et al. [2005] with a technique that searches the image for features according to an interest point operator and tracks regions where such features are absent. Tanaka et al. [2008] subdivide the screen according to a rectangular grid and compute viewability as a weighted combination of RGB values, variance of saturation in HSV color space, and variance of luminance in YCbCr color space for every grid

→ Diapo 14

cell. An annotation is then placed in a grid with a low viewability value. Grasset et al. [2012] analyze the salience of the video input and place labels in areas of low saliency, where they are unlikely to cover up important real-world information. For example, labels may be placed in the sky (Figure 7.11).

→ Diapo 15

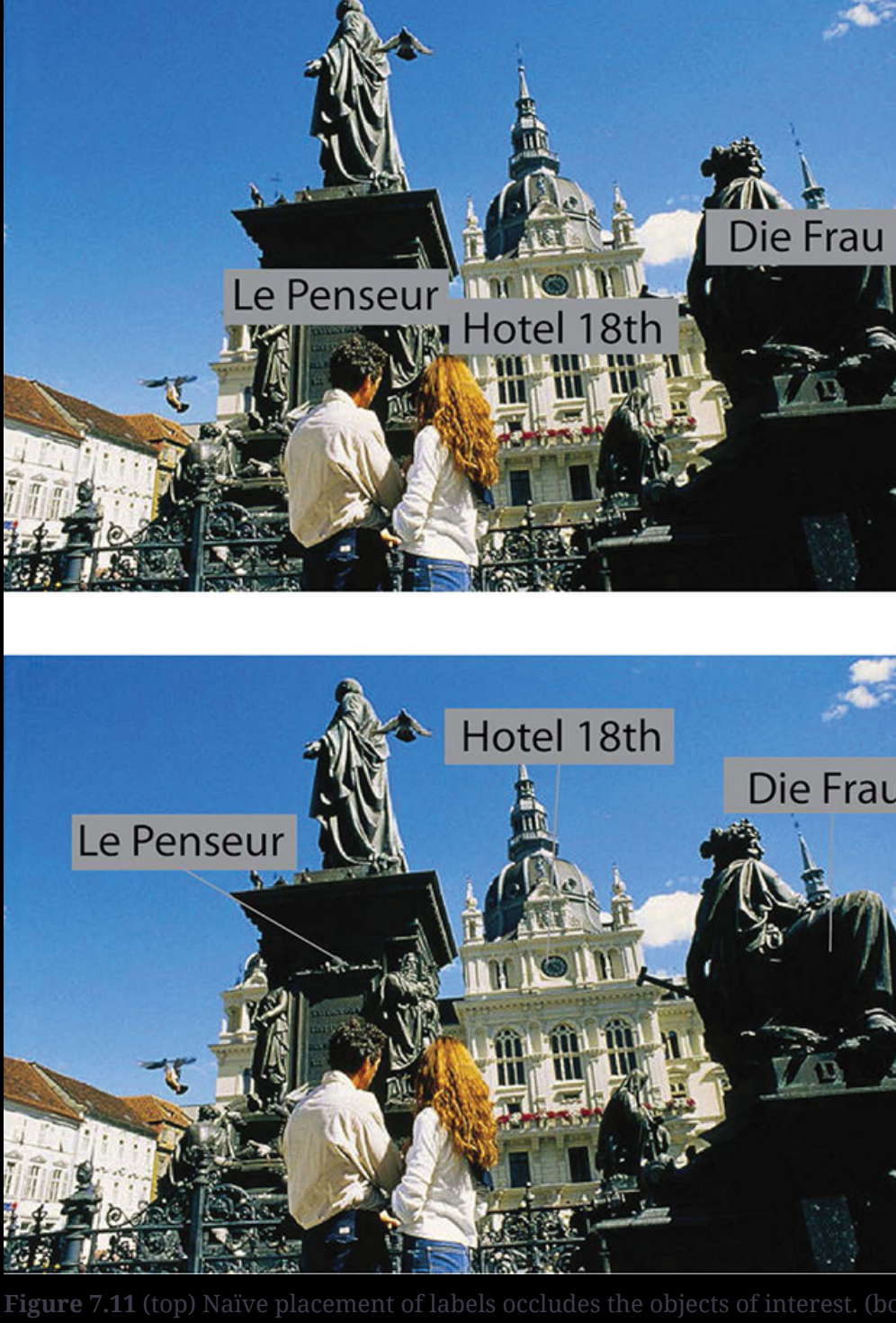


Figure 7.11 (top) Naïve placement of labels occludes the objects of interest. (bottom) Saliency-guided placement avoids occlusions and prefers empty areas, such as the sky. Courtesy of Raphael Grasset.

Legibility

LEGIBILITY

Overlaid information should be easily understandable on a cognitive level. However, we need to ensure not only the availability of cognitive support, but also **legibility**. By legibility, we mean low-level perceptual factors, such as sufficient size and contrast of the overlaid information.

First and foremost, legibility is a concern with textual labels, which must be read by the user. It is also relevant for all kinds of graphical information. Placing computer-generated information over a live video background can be problematic, if the background is very salient, uses strong colors, or includes textures containing high frequencies. This undesirable situation is further exacerbated in optical see-through systems, where a fully opaque overlay cannot be obtained, such that the maximum color fidelity is limited.

If the full extent of real-world objects and their visual properties appearing in the scene is known in advance, view management can determine the optimal placement of certain elements such as labels and largely avoid situations of poor legibility.

The legibility of text over various backgrounds in an optical see-through display was investigated by Gabbard et al. [2007]. They conducted a series of user studies and found that—as might be expected—user performance in search tasks is significantly affected by background texture and text drawing style. Gabbard et al. considered text placed on a uniformly colored billboard, which occludes the background, as well as text emphasized with outlines and drop shadows. The contrast achieved between the background and the emphasis color (i.e., the color of the billboard, outline, or shadow) as well as the contrast between the emphasis color and the text color significantly determine legibility. Note that the use of billboarded text may not be possible in scenarios that do not allow larger areas of the background to be occluded.

While an earlier study by the same group [Gabbard et al. 2006] showed good outcomes when saturated green was used as an emphasis color, the best results in dynamic environments would be achieved by adapting the emphasis color to the background. For example, Mendez et al. [2010] ensure that a minimum difference in visual saliency between foreground objects (such as labels) and background objects is achieved.

X-Ray Visualization

An important application of AR is to reveal hidden artifacts by artificially removing occlusions from real-world objects. This so-called **X-ray visualization** resembles the superhuman ability to look through solid matter—a favorite superpower in science fiction. A synthetic view of hidden artifacts is shown in the context of a real scene, so that an observer can infer spatial and semantic relationships between visible and hidden objects. Naïvely superimposing a virtual object on top of a real scene (Figure 7.12, top) does not lead to satisfactory results, as fundamental rules of depth perception are lost. Likewise, uniformly applying transparency to the virtual object (Figure 7.12, bottom) does not resolve this problem—Buchmann et al. [2005] showed that uniform transparency destroys spatial relationships. Moreover, it can lead to display clutter.

→ Diapo 21

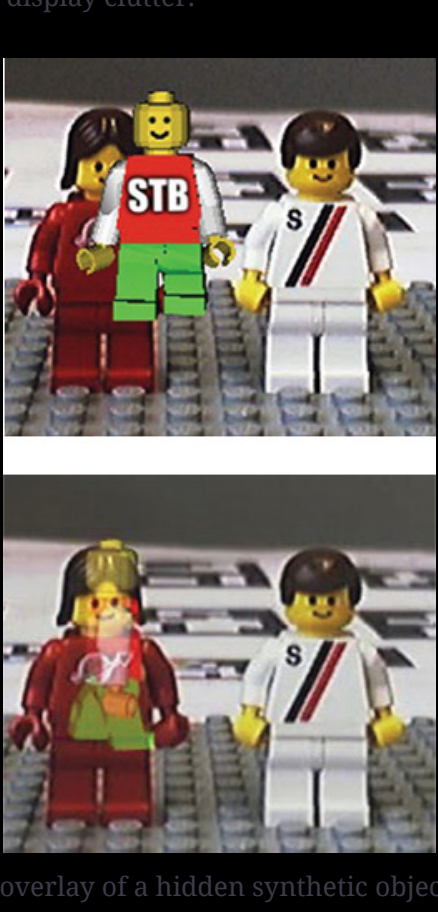


Figure 7.12 (top) Naïve overlay of a hidden synthetic object on top of a real scene. (bottom) Overlay of a synthetic object with uniform transparency. Courtesy of Denis Kalkofen.

A better way to produce X-ray visualizations is to render the object causing the occlusion with adaptive transparency, with the goal of revealing enough of the occluded object underneath so that it is easily comprehensible, while the major structural properties of the occluder are mostly preserved. Such a representation is often called **ghosting** [Feiner and Seligmann 1992] in computer-based illustration.

Ghostings from Object Space

Ghostings in AR can be produced by extending the phantom rendering approach presented in Chapter 5. Standard phantom rendering merely discriminates fully visible from fully occluded pixels through the use of a z-buffer. In contrast, a ghosting can be produced by determining for every occluded pixel of a virtual object the degree of occlusion (or transparency) of the occluder, which is then used for alpha-blending the occludee and the occluder.

For example, a ghosting can be generated by setting the transparency of the fragment based on a linear function of the object's principal curvature. For a given phantom, curvature can be precomputed and stored per vertex or as a texture. The curvature is then looked up or interpolated per fragment and converted into transparency after applying some scale and bias. The rationale of this approach is that strongly curved portions of the phantom's surface define its shape and should be more opaque, while flat areas can be made more transparent.

Another attribute that can be used to steer transparency is the dot product of the surface normal and the viewing direction. When the surface normal is perpendicular to the viewing direction at the silhouette of a phantom object, the dot product is 0. In contrast, for surfaces viewed in the direction of the normal vector, the dot product is 1. This property can easily be converted into a transparency value. Silhouette and principal curvature are complementary in terms of the aspects of an object's shape that they convey and can be used together.

Ghostings from Image Space

All attributes computed in object space, such as silhouette or curvature, generally suffer from imperfections of registration. Any mismatch in the pose of a real object and its phantom will lead to a displacement of the local effects derived from the phantom geometry, and these displacements will be disturbing to the viewer. Moreover, in AR, several (or all) of the real objects in a scene are often unknown and, consequently, cannot be used as phantoms. The only available information concerning these real objects may be their representation in the video image. However, we can make use of the assumption that we would like to show hidden virtual objects and treat everything that is visible in the video frame as an occluder.

To do so, we can try to identify the **shape hints** in the video image that convey the shape of the visible objects and set per-pixel transparency corresponding to the pixel's importance as a shape hint. One simple way of finding shape hints is to use an edge detector. After identifying which pixels of an image contribute to edges (e.g., using Canny edge detection), the edges can be emphasized. For example, after rendering a basic image with the video image as the background and virtual objects superimposed on top of it, the edges can be rendered in an opaque color on top of the basic image, resulting in a stylization similar to hidden-line graphics. In this way, a coarse ghosting is produced, which preserves some important depth relationships between real and virtual objects.

Artificially colored edges are a strong visual distraction, however, and extracting all edges from the entire video frame may lead to excessive clutter. Suppressing clutter is a general requirement for all techniques that rely on emphasizing shape hints. A reasonable approach to prevent excessive use of shape hints is to limit them to the areas of the image surrounding the virtual objects. These areas can be determined either manually—for example, by letting the user place a magic lens [Mendez et al. 2006]—or by computing an area from the screen projection of the virtual object. Rendering the virtual objects to a stencil or id-buffer and applying a distance transform to this buffer yields such a region with a chamfered boundary, which can be used to determine transparency of the shape hints—in our case, the edge pixels (Figure 7.13). Computing the distance transform can be done on the GPU [Rong and Tan 2006].

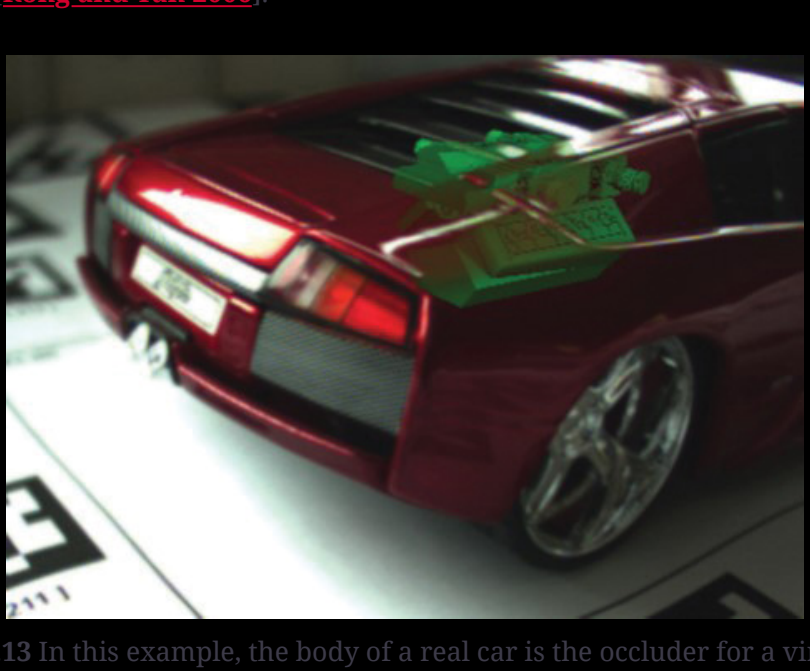


Figure 7.13 In this example, the body of a real car is the occluder for a virtual engine. After extracting contours as important shape hints, a 2D distance transform is applied to make the occluder seem more solid. Courtesy of Denis Kalkofen.

Edges extracted from the image are effective in conveying shape but have a tendency to lead to clutter and cannot convey the impression of a dense occluder. If such a dense occluder is acceptable or even desirable, a distance transform can be applied to the edge image and used to determine transparency. This operation will lead to larger areas of occlusion around the edges, with transparency gradually increasing in the direction away from the edge. The resulting visualization resembles an image produced by deriving transparency from curvature, as suggested earlier.

Edges are important shape cues but do not represent the only valuable information for supporting spatial perception. In general, we want to preserve visually **salient** image features, which are often defined as the joint effect of contrast measures found in attributes such as hue, luminosity, orientation, or motion. Salient features have been found to attract a viewer's gaze [Treisman and Gelade 1980].

Saliency can be computed from an image by analyzing its contrast on the levels of an image pyramid and adding up the effects across the levels of the pyramid [Itti et al. 1998]. To do so, the image is concerted to a color space such as L*a*b*, which directly encodes luminosity and red–green and blue–yellow color opponency. In this representation, an image pyramid is computed. Central differences are determined for every pyramid level and every channel of the L*a*b* image, and all differences are combined. Motion can be considered by computing the changes in luminosity over time. All of these computations can be efficiently performed in a fragment shader.

Sandor et al. [2010b] use this saliency computation to determine transparency of a ghosting. The transparency level for a fragment is computed from the difference between the saliency of the occluding image and the saliency of the occluded image.

In contrast to the approch of determining saliency per pixel proposed by Sandor et al. [2010b], Zollmann et al. [2010] determine saliency per area, based on a superpixel segmentation of the image. This strategy has the advantage that properties of consistent regions in the image can be exploited for the ghosting. Texturedness is one such property of a region that provides an important additional measure of saliency. Moreover, transparency can be determined per region (Figure 7.14). Because the regions are determined by segmenting natural boundaries in the video image, no additional spatial frequencies are introduced by the transparency modulation, leading to a less cluttered result.

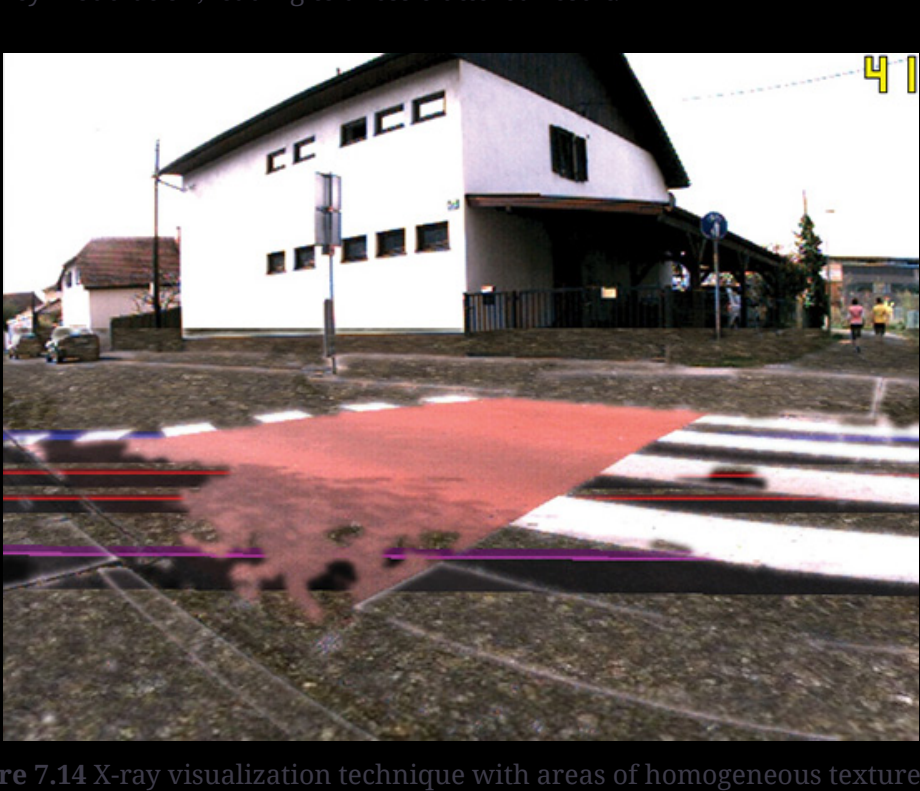


Figure 7.14 X-ray visualization technique with areas of homogeneous texture assigned to a particular level of transparency to achieve a consistent appearance. Courtesy of Stefanie Zollmann.

Implementation with G-Buffers

The implementation of ghostings is based on geometric buffers (G-Buffers) known from non-photorealistic rendering [Saito and Takahashi 1990]. In a three-pass rendering approach, the first pass renders the individual objects of the scene into a set of buffers, the second pass applies image processing techniques to the buffers, and the third pass composes the final result from the individual buffers by sweeping over the buffers in depth order.

Buffer Rendering

The rendering uses multiple G-Buffers, each containing an approximation of scene objects belonging to a particular group. Using this technique, we can isolate the styling applied to different groups, while the collection of all G-Buffers approximates the whole scene. During buffer rendering, we use the regular rendering pipeline to extract all the necessary information that will be used during the processing of the buffers. The scene is traversed in a single pass with multiple render targets—namely, multiple G-Buffers. Every object is rendered to exactly one of the G-Buffers, which is determined by its group membership (Figure 7.15).

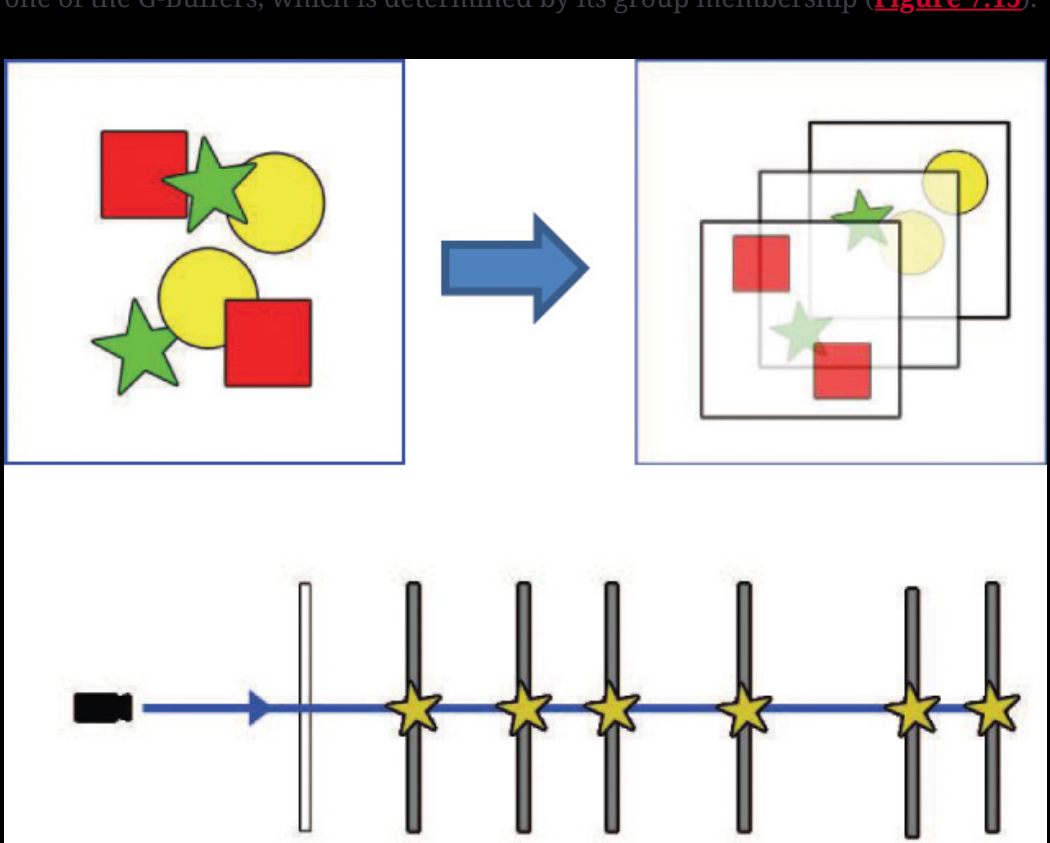


Figure 7.15 (top left) An illustration of a scene. (top right) One possible G-Buffer volume. Notice that the G-Buffer does not represent a depth layer. (bottom) Composition by front-to-back traversal of the G-Buffer volume. Courtesy of Erick Mendez and Denis Kalkofen.

Buffer Processing

For every buffer, image processing techniques can be applied to compute additional information—for example, to detect edges or regions with high curvature, to extract regions with particular color or depth values, or to mark a particular region supplied interactively by the user. Some techniques consider more than just the fragment's value, such as its neighborhood, fragment values of other buffers in the same G-Buffer, or fragment values from different G-Buffers. In this way, multiple additional image components containing auxiliary information can be added to the G-Buffer.

Scene Compositing

In the final compositing step, the information from the set of G-Buffers is merged into a final image using a front-to-back traversal of the G-Buffers (Figure 7.15, right). Note that simple blending of the G-Buffers is not enough: Per-pixel occlusions are essential for the desired effects. Since the G-Buffers are already available in view coordinates and in screen resolution, the problem is reduced to sorting the depth components of the G-Buffers. Once this sorting has taken place, we proceed to combine all of the fragments into a single output. The compositing rules used during this process can arbitrarily alter the contribution of a particular pixel from one of the G-Buffers. For example, color or transparency of a particular pixel may be modified based on the importance of the pixel that was visited along the ray before the current one.

Spatial Manipulation

If a scene is densely populated with objects, and several visible and hidden objects that are in close proximity must be shown simultaneously, a ghosting may not be sufficient to uncover all relevant information. Instead, it may be more instrumental to trade unoccupied screen space for an increased visibility of relevant objects. Spatial manipulation of the objects in the scene achieves precisely that end—in AR, we can rearrange the scene. We can create the illusion that either some objects have moved or otherwise changed to make room for hidden objects. Both the moved object and the revealed object could be either real or virtual. In this section, we take a closer look at two types of spatial manipulations: explosion diagrams and space distortions.

Explosion Diagrams

Explosion diagrams originate in hand-drawn technical illustrations as a technique to present object assemblies. The key idea of explosion diagrams is that the parts of a complex object are arranged in such a way that it becomes possible to mentally reassemble the object. In that way, usage of the available screen space for showing visual information on the parts is maximized, while the perception of the object's spatial structure is still well supported. Finding a suitable layout of the exploded parts relative to each other is therefore essential in the generation of successful explosion diagrams.

When using explosion diagrams as a visualization technique in AR, finding the right arrangement of parts is not the only challenge that must be addressed: AR also requires that parts from the real world be convincingly relocated. To achieve this goal, visual information must be transferred from the original location to the destination location (Figure 7.16). After this transfer of visual information, the original location must be filled with new imagery. Finally, correct mutual occlusion between virtual and real must be resolved.

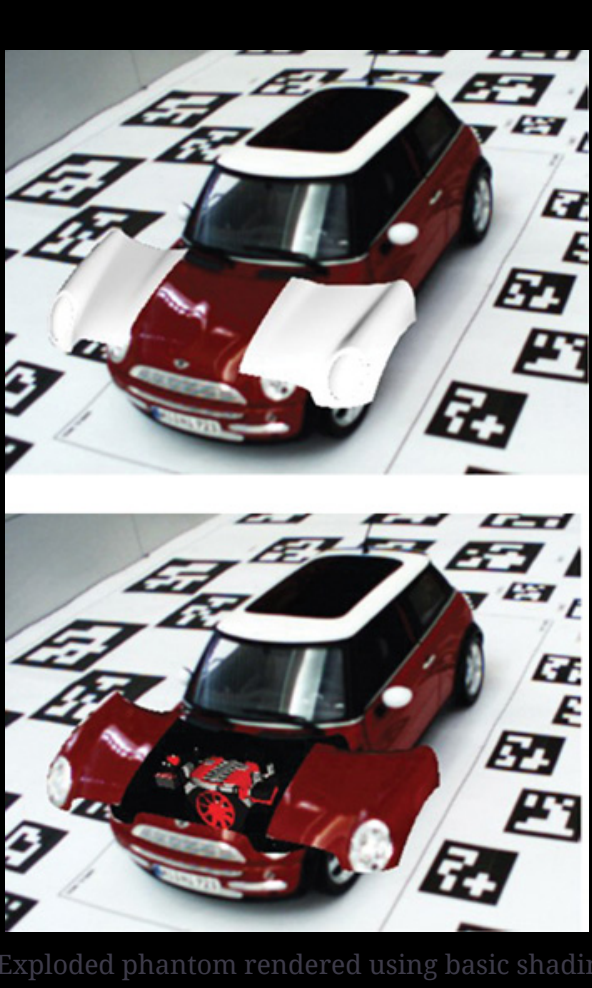


Figure 7.16 (top) Exploded phantom rendered using basic shading. (bottom) With video-textured dual phantom rendering, visible pixels are exploded and uncovered background pixels are colored black. Courtesy of Markus Tatzgern and Denis Kalkofen.

For the displacement of real world parts, a **video-textured phantom** can be used [Kalkofen et al. 2009] [Tatzgern et al. 2010]. This is a virtual representation of a real-world object, which is texture mapped with the live video image. Projective texture coordinates are computed for each surface point of the video-textured phantom from a viewpoint corresponding to the current camera image; that is, each vertex of the phantom is transformed to the original vertex position which it occupied before displacement.

This computation can easily be performed in a vertex shader and applied to the geometry of the phantom object after it has been displaced to a new location. Video-textured phantoms are rendered with texture mapping, but without any shading, since we can assume that illumination effects are already present in the video texture of the real object, and only a single rendering pass is required. Therefore, the rendering of video-textured phantoms is computationally very inexpensive. Moreover, this technique works for zero displacement and results in the same effect as when using a conventional phantom rendering approach. This property makes it straightforward to produce animated explosion diagrams, which start from an unmodified real object.

Video-textured phantoms enable us to move real-world parts to new locations on the screen and reveal hidden objects at the original screen location. However, the uncovered area will usually not be fully occupied by virtual objects. The pixels that are not occupied by the virtual object should not show the original video image, but rather should show a background color. To this end, a second rendering pass is executed, which initializes unoccupied pixels from the uncovered area with the background color. This **dual phantom rendering** algorithm may be expressed as the following:

1. Switch to render target T_1.

2. Clear render target T_1.

3. Render video-textured (displaced) phantoms to T_1.

4. Render virtual objects to T_1.

5. Switch to render target T_2.

6. Fill T_2 with the current video image.

7. Render phantoms at their original location in background color.

8. Superimpose T_1 on top of T_2.

If multiple phantoms overlap in screen space, dual phantom rendering may not yield correct results, as shown in Figure 7.12, top. In such a case, video information from one undisplaced phantom is transferred to another displaced phantom, so it appears twice in the frame. To suppress this artifact, it is necessary to only copy those pixels from the video that are visible in the original video frame. This goal can be achieved by first rendering phantoms to an id-buffer. For every fragment of a video-textured phantom, the modified algorithm first determines whether the corresponding fragment at the original location is visible, before writing the information from the video frame to the new location. If the fragment is not visible at the original position, a replacement color, synthetic shading, or inpainting must be used (Figure 7.17). This algorithm is called **synchronized dual phantom rendering**:

1. Switch to render target T_1 (id-buffer).

2. Clear render target T_1.

3. Render all phantoms to T_1.

4. Switch to render target T_2.

5. Fill T_2 with the current video image.

6. Fill those pixels in T_2 that correspond to the phantom to be displaced with the background color.

7. Render video-textured (displaced) phantoms to T_2, relying on T_1 to control usage of the video image.

8. Render virtual objects to T_2.

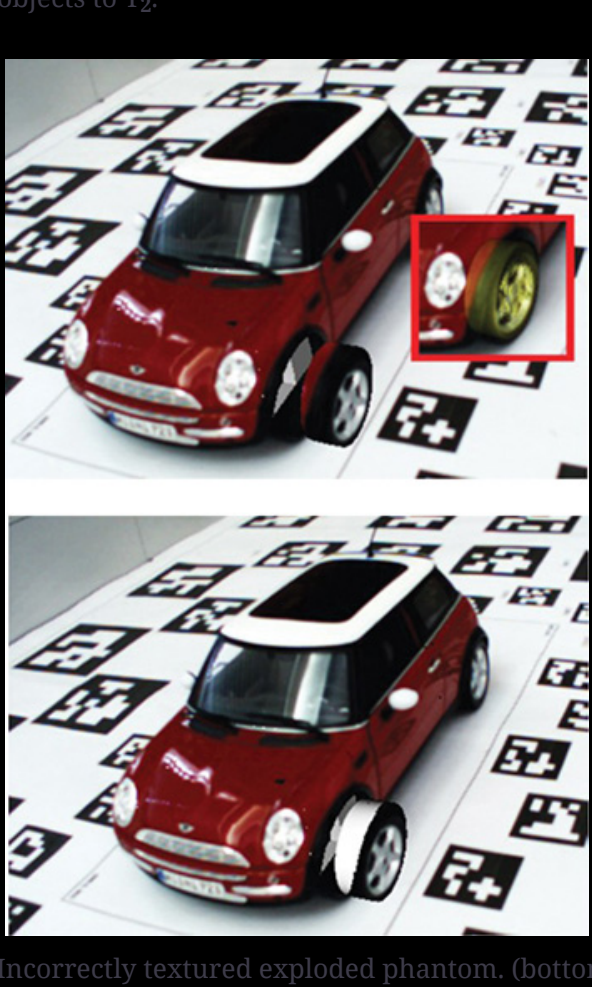


Figure 7.17 (top) Incorrectly textured exploded phantom. (bottom) Synchronized dual phantom rendering can identify pixels that cannot be video-textured and apply a different rendering style to these pixels. Courtesy of Markus Tatzgern and Denis Kalkofen.

Space Distortion

Sometimes hidden information should be revealed, but there is no unoccupied space, so an explosion diagram cannot be applied. However, a video-textured phantom might be able to be scaled down to occupy less screen space, thereby making room to display a hidden object. This idea is applied in the **melt** visualization technique proposed by Sandor et al. [2010a]. In their example, occluding buildings in an outdoor environment are scaled down in vertical directions to unveil hidden objects. The scaling generates the impression that these buildings are "melted." Sandor et al. point out that this technique is particularly effective when there are multiple layers of occluders, because it avoids the clutter of conventional X-ray visualization.

Variable perspective view [Veas et al. 2012b] is an AR visualization technique developed to combine views from different perspectives in a single image. It combines two virtual cameras, the main camera (mc) and the secondary camera (sc; also known as the far camera). This method applies a skinning algorithm for skeleton animation, as shown in Figure 7.18. It uses a single joint with two bones with the following parameters: *d* is the distance to the rotation axis (the distance from the main camera to the joint), α is the angle of rotation, and ϕ is the area of effect (where the rotation will be interpolated).

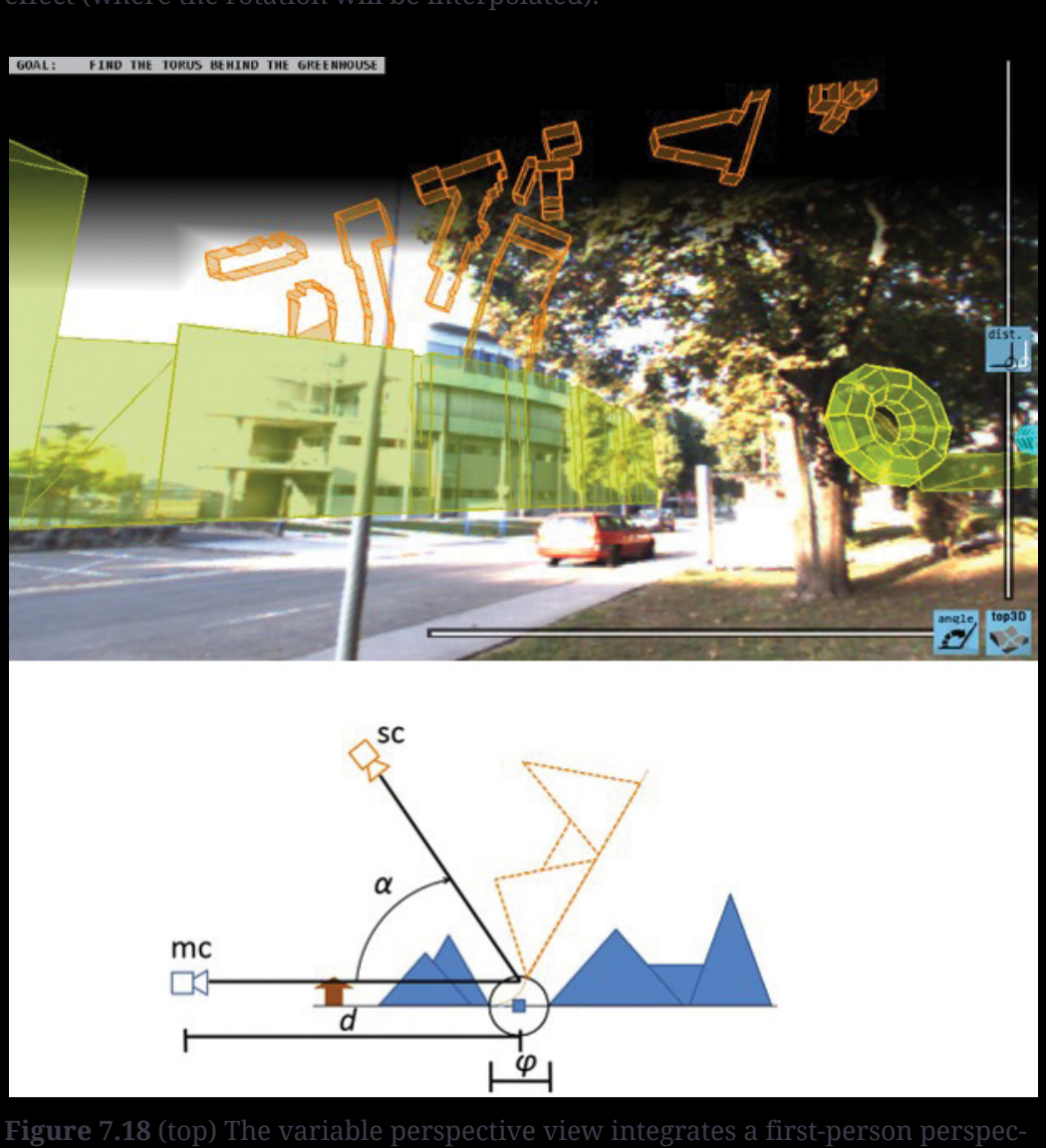


Figure 7.18 (top) The variable perspective view integrates a first-person perspective of the foreground and a virtual top-down view of the distant scene in one image. Courtesy of Eduardo Veas. (bottom) The variable perspective view is achieved by applying principles from skeleton animation to the tilted part of the scene geometry.

All vertices in the virtual scene are weighted according to their distance from the main camera to the rotation axis. The weight of vertices defines whether they fall in the view of the main camera, in the view of the secondary camera, or in the transition zone, where they are interpolated. To further extend the overview, the secondary camera is placed at a distance from the AR view, enabling it to capture more information from the digital data, while correctly registering the video for real-world context.

Information Filtering

A notorious problem of information display in AR is that too much information leads to display clutter, thereby hindering—rather than assisting—a user in understanding the environment. Techniques intended to reduce the amount of augmented information presented to a reasonable volume are generally called **information filtering**. Information filtering can be achieved by either of the following two strategies: The filtering can be based on knowledge about the importance of the information to the application or user (*knowledge-based* filter), or it can be based on *spatial* considerations, supplied either in screen space or in object space. In this section, we investigate these two possibilities and give examples of their application.

Knowledge-Based Filter

A knowledge-based filter for AR was described by Feiner et al. [1993]. KARMA (Knowledge-based Augmented Reality for Maintenance Assistance) automatically creates technical illustrations suitable for AR based on a set of communicative goals provided to an illustration generator, the intent-based illustration system. This generator uses a rule-based approach to synthesize illustrations. Internally, it tries to satisfy a set of constraints resulting from the communicative goals (such as the goal to show a particular object) and the current real-world situation (such as which objects are visible from the current viewing position of the user). KARMA represents knowledge in the form of rules that specify which kind of illustration (or illustration style) can satisfy a particular constraint. The process of synthesizing a particular illustration, therefore, becomes a procedure of searching for and backtracking to the right application of rules from the available set.

KARMA's illustration engine was originally conceived to deliver illustrations for VR, but it was extended to cover AR specifics by supplying certain constraints that the system cannot alter, such as the position of physical objects. Figure 1.4 shows an example of superimposed maintenance instructions for a laser printer viewed through an optical see-through HMD.

Spatial Filter

Spatial filtering uses geometric or geographic information, which is directly derived from the AR scene representation, to reduce the amount of information. A simple example would be a labeling algorithm (refer to the section "Annotations and Labeling") that labels objects in order of increasing distance from the observer and terminates the process when it runs out of available screen space.

Oftentimes the spatial filtering is combined with a degree of interactive control. A classic example is a **magic lens** technique. Originally introduced by Bier et al. [1993] as a 2D user interface widget, a magic lens is a region of a 2D or 3D environment that is rendered in a different style than its surrounding. Typical uses of magic lenses include enhancing data of interest, revealing hidden information, and suppressing distracting information.

In 3D, magic lenses can be defined as a region either in screen space or in object space [Viega et al. 1996]. Magic lenses in screen space—called flat magic lenses—affect every object that has a projection falling into the area covered by the magic lens. Magic lenses in object space—called volumetric magic lenses—affect every object inside the lens region. Both types of lenses can be used in AR to limit certain types of augmentations or annotations to an area of interest.

For example, Looser et al. [2007] discuss the use of a handheld magic lens for object magnification, object selection, and discovery of hidden information. In their approach, a user wearing an HMD holds a tracked wand in the hand. The tip of the wand is coupled to the magic lens, so it can be conveniently placed in the environment.

Bane and Höllerer [2004] present another type of spatial filter, the X-ray tunnel (Figure 7.19). The tunnel defines a bounded region in screen space, extending it from the user's position in the direction of view. It is essentially a special type of magic lens, which is suitable for uncovering hidden structures at a distance. This kind of visualization resembles looking down a tunnel into an inner portion of the scene. Inside the tunnel, the region closest to the observer is empty, while a middle region displays objects in wireframe to establish some context. Finally, in the region of interest at the far end, the virtual object is rendered and a uniformly colored background is displayed to ensure sufficient contrast. Depth perception is supported by displaying "rails" that give a perspective foreshortening cue and by numerical display of the distance to the hidden object. Users can opt to display semantic entities, such as interior rooms, in their entirety rather than depicting arbitrarily sliced geometry. This can aid hidden scene understanding significantly.

TUNNEL INFO
Distance: 25.755208
Focus Length: 2.000000
Context Length: 2.000000
Zoom Factor: 1.000000

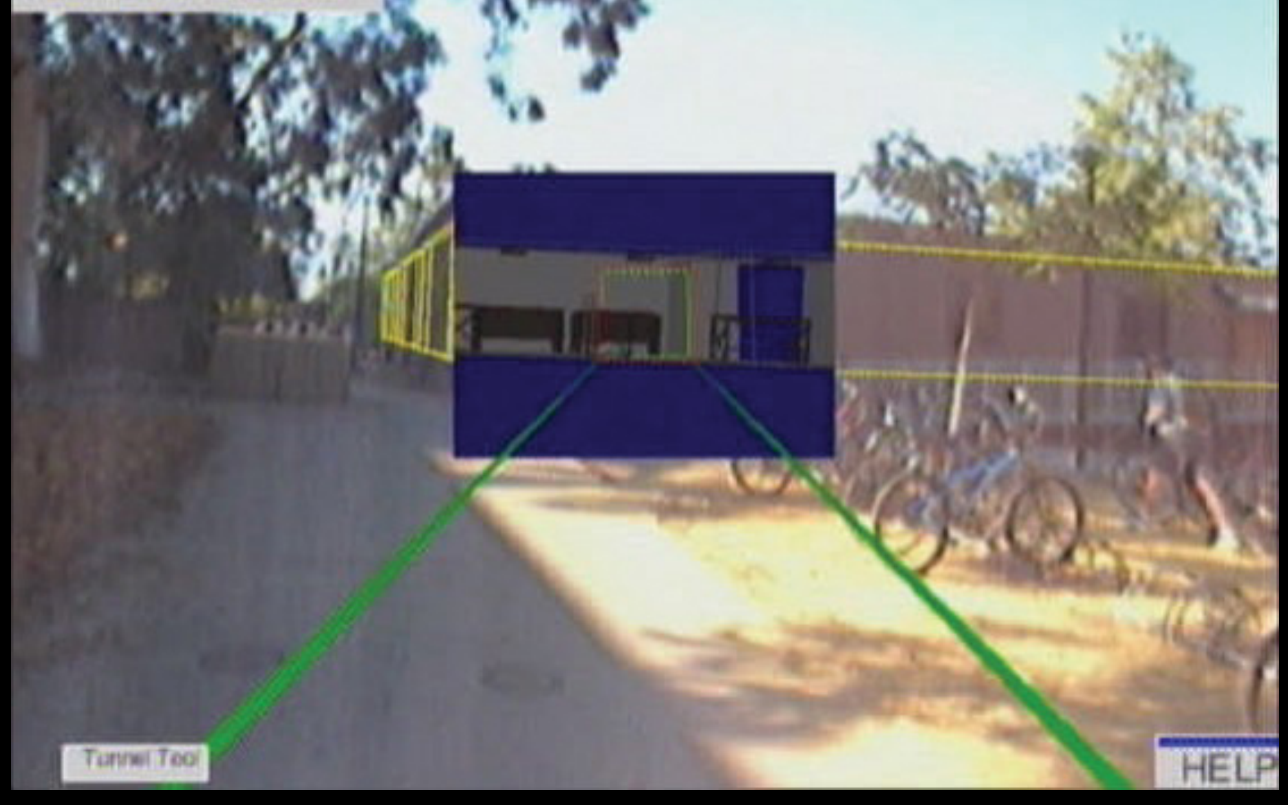


Figure 7.19 The X-ray tunnel enables a user to look inside a building and gives hints about the distance of the uncovered geometry.

Mendez et al. [2006] describe the use of magic lenses for context-driven visualization in AR (Figure 7.20). The multipass rendering technique they discuss can render arbitrary sets of concave lenses. Moreover, these lenses can be associated with special effects (e.g., ghosting effects) that are dynamically applied based on properties of an object. The properties are supplied by the application as markup information attached to scene objects. This approach makes it very easy to combine certain types of magic lens effects and apply effects selectively. For example, an effect could remove certain unimportant objects from the scene, while important, but occluding, objects are rendered using a transparency effect.

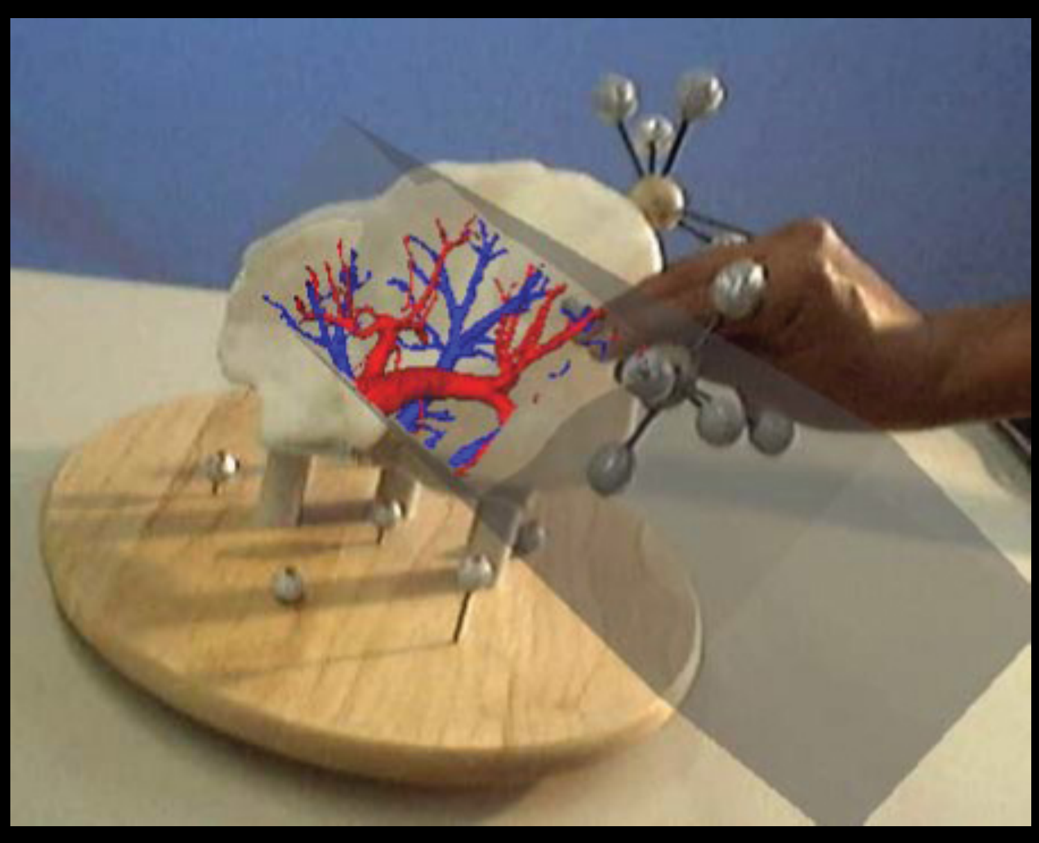


Figure 7.20 A magic lens reveals the vessels inside a liver model. Courtesy of Erick Mendez and Denis Kalkofen.

Combined Knowledge-Based and Spatial Filter

The focus–nimbus technique proposed by Julier et al. [2002] represents a hybrid knowledge-based and spatial filtering strategy. These researchers argue that the level of detail of information provided for a specific object in the scene should consider both the importance of an object for the user's current task and the proximity to the user. Annotations for an object that is too unimportant or too far away are faded out. To compute this combined measure, the user's current task and position are used to define a focus region, and the attributes of every object define a nimbus region. Annotations for the objects are displayed to a degree that is determined by how much focus and nimbus intersect. Task-specific attributes influence the extent of focus and nimbus, such that an object that is highly relevant for the user will have a larger nimbus. Julier et al. describe the application of their technique in an urban warfare scenario (Figure 7.21).

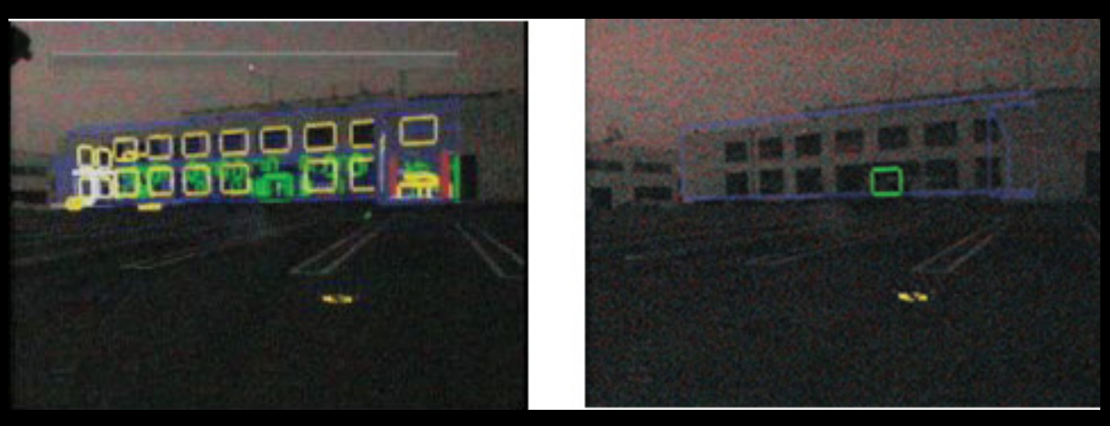


Figure 7.21 (left) Unfiltered overlay of the available information produces a cluttered display. (right) By applying the focus–nimbus filter, the overlays are limited to information of high-enough relevance—in this case, the building outline and entrance. Courtesy of Simon Julier.

Any form of filtering, including the hybrid focus–nimbus approach, leads to the suppression of potentially relevant information. Ideally, less relevant information should be redacted rather than suppressed. In keeping with this aim, Tatzgern et al. [2016] proposes to aggregate related data points by clustering, rather than removing data with a filter. Clustering has the advantage over filtering that the complete information space is preserved. Some commercial AR browsers cluster data uniformly to control clutter, although this does not necessarily lead to a better overview and does not scale to large data sets. Tatzgern, however, proposes creating an information hierarchy by recursive clustering of annotations. This representation is conceptually similar to semantic level of detail. The clustering combines user-controlled spatial attributes (e.g., distance) with nonspatial attributes (e.g., semantic tags). The weighted sum of these attributes provides a ranking of the data's relevance for the user.

To avoid visual clutter, a display algorithm shows data that is relevant for the user in more detail, while adapting the overall amount of information shown to the available screen space. It does so by solving an incremental optimization problem, deciding which nodes in the hierarchy are selected for display. Users can interactively adjust priorities to drill down on data deemed relevant, and reveal all available details on demand (Figure 7.22).

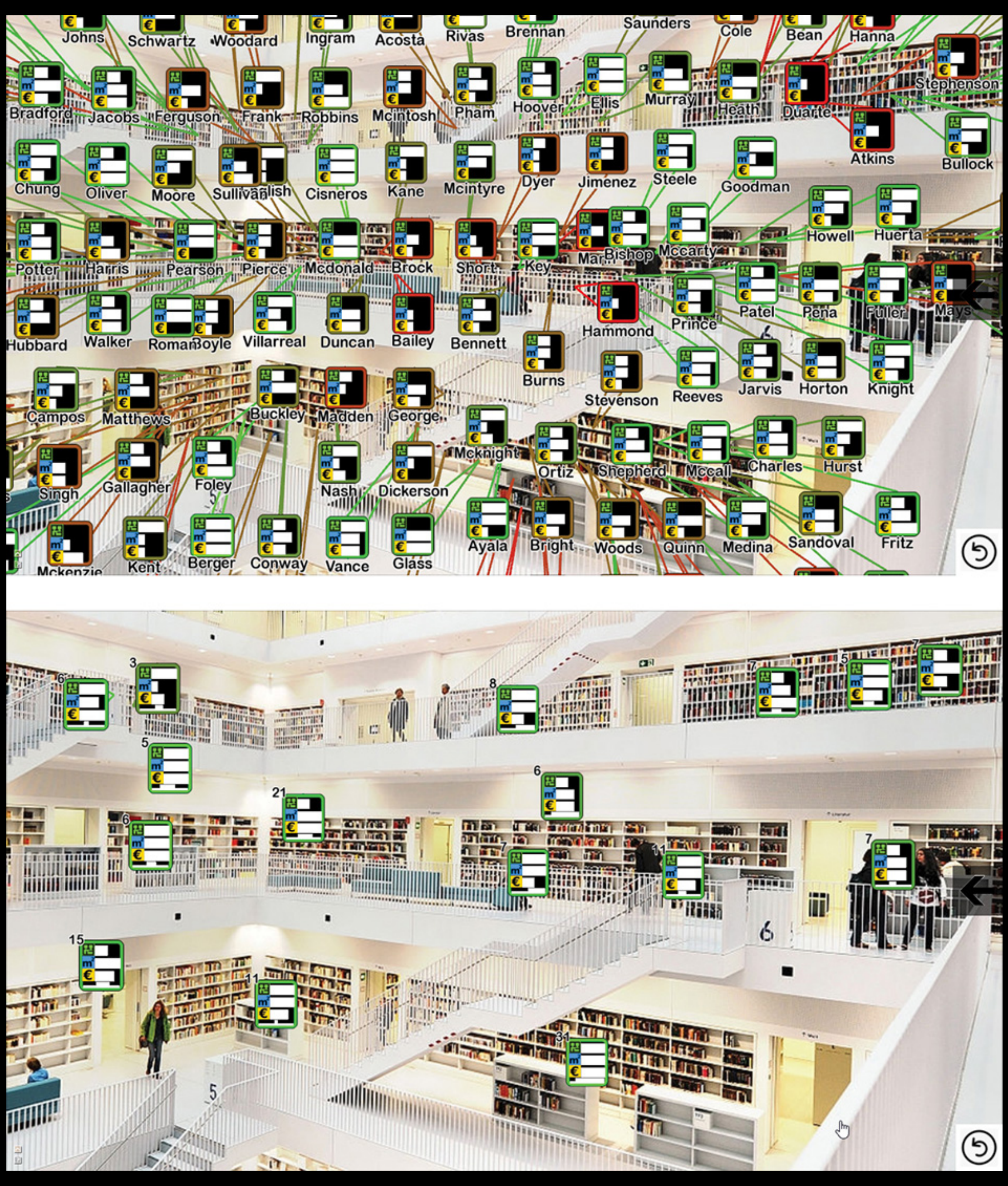


Figure 7.22 Icons representing the location of books selected in a library. (top) Showing all books matching to a query at once leads to clutter. (bottom) Clustering the books by similarity and displaying only clusters for groups of books produces an easily comprehensible view. The user can click on the clusters to further unfold them. Courtesy of Markus Tatzgern and Denis Kalkofen.

Summary

Visualization determines how information should be shown. In AR, this primarily means determining which additional information should be shown, since the user is already perceiving all information present in the real world. Situated visualization is a principle of visualization for AR where information is attached to real-world objects. Annotation and labeling techniques are necessary for this kind of visualization, to avoid illegible layouts and cluttered displays. Another important principle for AR is the use of X-ray visualization to uncover hidden or otherwise imperceptible structures—for example, through the use of ghostings or explosion diagrams. Finally, information filtering techniques such as magic lenses are useful in delivering the right amount of task-dependent information to the user.